GIT — Complete Study Notes

Course: Comprehensive Git Source Control | Instructor: Jason Taylor
Format: Phase-wise teaching document for DevOps interview preparation

PHASE 1 — THE MENTAL MODEL

What is Git and Why Does It Exist?

Git is a distributed version control system (VCS) created by Linus Torvalds — the same person who created the Linux kernel. His motivation was to build a VCS capable of handling the sheer scale of the Linux project: over 15 million lines of code, more than 1,200 worldwide contributors, and dozens of active branches simultaneously.

Version control solves a real problem: when you create anything — source code, design files, web content — you need the ability to track every change, know who made it, when, and why, and be able to go back to any point in history. Without version control, the typical workaround is creating copies like "project_v1", "project_v2_final", "project_v2_FINAL_FINAL" — which is fragile and unscalable.

Why Git Specifically?

- **Distributed** — every developer has a full copy of the repository, including all history
- **Local operations** — most commands (commit, diff, log, branch) work completely offline — only push/pull/clone need a network
- **Speed** — because most operations are local, Git is very fast
- **Free and open source** — zero cost, huge community, tons of resources
- **De facto standard** — integrates with editors, CI/CD systems, bug trackers, and platforms like GitHub, GitLab, Bitbucket

⚠ NOTE: Git is DECENTRALISED — even though most teams use a central server (GitHub), that server is just another Git repository. There is no master database. Everyone has a full copy.

The Three Local States of Git

Every file in a Git-managed project lives in one of three local states at any given time. Understanding this is the single most important mental model for Git.

State	Also Called	What It Means	How To Move To Next State
Working Directory	Working Tree	Where you edit files. Git watches this folder for changes.	git add <file>
Staging Area	Index / Cache	A holding area. You choose which changes go into the next commit.	git commit

Repository	Commit History	Permanent record of all commits. Lives in the hidden .git folder.	git push (to remote)

A fourth conceptual state is the Remote Repository — another Git repository (on GitHub, GitLab, etc.) that teams share. It is internally just another full Git repository with its own three states. Commands that touch the remote: git clone, git push, git pull, git fetch.

Key Terminology Reference

- **Repository** — the .git folder containing all version history and configuration
- **Working Directory** — the project folder on your computer where you edit files
- **Staging Area** — a preparation zone — files queued here will be included in the next commit
- **Commit** — a permanent snapshot of staged changes, with a unique SHA-1 ID
- **SHA-1** — a 40-character unique identifier for every commit (first 6-7 chars usually sufficient)
- **HEAD** — a pointer that always points to the last commit on the currently active branch
- **origin** — the conventional name for the remote repository (set automatically by git clone)
- **master / main** — the default branch created when a repository is initialised
- **Branch** — a lightweight label / pointer to a specific commit — creates a timeline for changes
- **Tracked file** — any file Git is actively monitoring — either previously committed, or added to staging
- **Untracked file** — a file Git sees in the working directory but is not yet managing

⚠ NOTE: Branches are just pointers. They are NOT folders or copies. Creating a branch costs virtually nothing in Git.

PHASE 2 — CORE BUILDING BLOCKS

Starting a Repository — Three Ways

Method 1: Fresh Project (git init)

Use this when you are starting from scratch with no existing code. Git creates a new .git folder in the directory you specify.

```
git init fresh-project      # creates fresh-project/ with .git inside
cd fresh-project
ls -al                      # shows the hidden .git folder
git status                  # shows branch, commit state, and untracked files
```

`git init fresh-project` → initialises a new Git repository in a folder named fresh-project

`ls -al` → -a lists ALL files including hidden ones like .git; -l is long listing format

`git status` → shows the current state of the working directory and staging area

⚠ NOTE: The .git folder IS the repository. Delete it with rm -rf .git and Git no longer manages the project. The source files remain untouched.

Method 2: Existing Project (git init in-place)

Use this when you have an existing folder with code and want to add Git version control to it.

```
cd web-project              # navigate into your existing project
git init                    # initialise Git in-place
git add .                   # stage all existing files recursively
git commit -m "Initial commit"
```

`git add .` → the period means "current directory" — stages all files recursively from here down

`git commit -m "Initial commit"` → -m allows inline commit message without opening an editor

Method 3: Clone an Existing Project (git clone)

Use this when joining a project that already exists on GitHub or another remote. Git downloads the entire repository — all history, all branches — and sets up the remote reference "origin" automatically.

```
git clone https://github.com/username/repo-name.git
cd repo-name
git status
git remote -v               # verify origin URL
```

`git clone <url>` → makes a full local copy of the remote repository

`origin` → the name automatically assigned to the remote you cloned from

`git status` → after clone shows: on branch master, up-to-date with origin/master, clean

The Basic Git Workflow (add → commit → pull → push)

This is the fundamental workflow you will use every single day. Every change goes through these four stages.

```
Edit file in working directory
         ↓
git add <file>              # move to staging area
         ↓
git commit -m "message"     # save to local repository
         ↓
git pull origin master      # sync with remote (best practice before push)
         ↓
git push origin master      # send to remote repository
```

git status — Your Constant Companion

git status is the first command you should run before doing anything. It shows you exactly where every file is in the three-state model.

```
git status
```

Possible output states:

- **nothing to commit, working directory clean** — everything is committed; no pending changes
- **Untracked files:** — files Git sees but is not managing yet

- **Changes not staged for commit:** — tracked files modified in working directory but not yet staged
- **Changes to be committed:** — files in the staging area, ready for the next commit
- **Your branch is ahead of origin/master by N commits** — local commits not yet pushed

Tracked vs Untracked Files

A tracked file is any file that Git is actively monitoring. This means either: (a) it has been committed to the repository at least once, or (b) it has been added to the staging area with git add. An untracked file is one that exists in the working directory but Git has never been told about.

```
git ls-files                # list all files currently tracked by Git
```

⚠ NOTE: git commit -a (the shortcut to add and commit in one step) ONLY works on tracked files. New files that have never been added to Git will be ignored by -a and left as untracked.

PHASE 3 — SYNTAX AND COMMANDS

git add — Staging Changes

git add moves changes from the working directory into the staging area. Think of the staging area as a shopping basket — you curate what goes in before you check out (commit).

```
git add <filename>          # stage a specific file
git add .                   # stage all changes from current directory down
git add -A                  # stage all changes + handle renames and deletions
git add -u                  # update index for tracked files only
(renames/deletes)
```

`git add .` → stages new files and modifications from current directory recursively

`git add -A` → stages everything including renames and deletions — safest option after OS-level file operations

`git add -u` → only updates Git's knowledge of already-tracked files (renames/deletes); ignores new untracked files

⚠ NOTE: You can add a file to staging, then modify it again in the working directory. git status will show the SAME file in BOTH "Changes to be committed" AND "Changes not staged for commit". The commit will only capture the version that was staged at commit time.

git commit — Saving to the Repository

```
git commit                  # opens default text editor for commit message
git commit -m "message"     # inline commit message (no editor)
git commit -am "message"    # shortcut: add tracked files + commit in one step
git commit --amend          # modify the LAST commit (message or content)
```

`git commit` → without -m, Git opens your configured text editor for the message

`-m "message"` → provides the commit message inline; use for short one-liners

`-a` → automatically stages all TRACKED modified files; does NOT add new untracked files

`--amend` → rewrites the last commit; useful for fixing typos in messages or adding a forgotten file

⚠ NOTE: --amend rewrites history. Never amend commits that have already been pushed to a shared remote repository — it will cause conflicts for other developers.

git pull — Fetching and Merging from Remote

```
git pull origin master      # fetch + merge remote master into local master
git pull --rebase origin master  # fetch + rebase local commits on top of remote
```

`origin` → the name of the remote repository (set by git clone)

`master` → the branch to pull from on the remote

`--rebase` → instead of creating a merge commit, replays your local commits on top of the downloaded commits — produces a linear history

Best practice: always run git pull before git push to avoid rejected pushes due to diverged history.

git push — Sending to Remote

```
git push origin master      # push local master to remote origin
```

`git push origin master` → sends all committed local changes on master up to the GitHub repository

⚠ NOTE: git push will fail if the remote has commits your local does not have. Always pull first.

Backing Out Changes

Git gives you an escape hatch at every stage. Here are all the ways to undo:

Scenario	Command	Effect
Unstage a staged file	git reset HEAD <file>	Moves file back to working directory (unstaged); file content unchanged
Discard working directory changes	git checkout -- <file>	Reverts file to last committed version; DESTRUCTIVE to uncommitted work
Unstage a staged deletion	git reset HEAD <file>	Unstages the deletion; file still missing from working directory
Restore a deleted file	git checkout -- <file>	Restores deleted file from last commit
Amend last commit	git commit --amend	Modifies last commit message or content

⚠ NOTE: git checkout -- <file> is DESTRUCTIVE. It discards uncommitted changes in the working directory with no way to recover them. Git does not put them in a trash bin.

Renaming and Moving Files

Git tracks renames as a rename only when it sees them correctly. There are three scenarios:

Using Git's mv command (preferred)

```
git mv level3-file.txt level3.txt    # rename using Git
git status                           # shows "renamed: old → new" already staged
```

```
git commit -m "Renaming file"
```

`git mv` → performs the rename at OS level AND tells Git — staged automatically

Using OS/bash mv command

```
mv old-name.txt new-name.txt          # bash rename (outside Git)
git status                            # Git sees this as deleted + new file
git add -A                            # -A reconciles the rename correctly
git status                            # now shows "renamed: old → new"
```

⚠ NOTE: If you rename a file outside of Git (via OS, Finder, etc.) and use just git add <newfile>, Git may see it as a deletion and a brand new addition rather than a rename. Always use git add -A or git add -u after OS-level renames.

Deleting Files

```
# Delete a tracked file using Git:
git rm <filename>                     # deletes file + stages the deletion
git commit -m "Deleting file"

# Delete a tracked file using OS (outside Git):
rm <filename>                         # bash delete
git add -A                            # Git now sees and stages the deletion
git commit -m "Deleting file"

# Unstage a deletion (change your mind):
git reset HEAD <filename>             # unstage; file still missing from disk
git checkout -- <filename>            # restore file from last commit
```

`git rm <file>` → removes file from working directory AND stages the removal in one step

`git reset HEAD <file>` → unstages the deletion; the file is still physically missing

`git checkout -- <file>` → restores the file from the last commit; undoes the deletion in the working directory

Viewing History (git log)

git log is how you inspect the commit history. It has many options to customise the output.

```
git log                               # full history, reverse chronological
git log --abbrev-commit               # shortened SHA-1 IDs
git log --oneline                     # one line per commit
git log --oneline --graph --decorate --all   # the power view
git log --since="3 days ago"         # date-based filter
git log -- <filename>                # history for a specific file
git log --follow -- <path>           # follow file through renames
git log <SHA1>..<SHA2>              # range of commits
git show <commit-id>                 # show changes in a specific commit
```

`--oneline` → condenses each commit to one line with shortened SHA-1 and message

`--graph` → shows an ASCII art graph of the branching/merging history

`--decorate` → adds labels for branches, HEAD, tags, and other references

`--all` → includes commits from all branches, not just the current one

`--follow` → traces file history across renames; without it, log stops at the rename

`git show <id>` → displays the full diff of what changed in that specific commit

Git Aliases

Aliases let you create short custom commands for long Git commands. They are stored in ~/.gitconfig under the [alias] section.

```
# Create an alias
git config --global alias.hist "log --all --graph --decorate --oneline"

# Use it
git hist

# View/edit aliases
cat ~/.gitconfig
# [alias]
#     hist = log --all --graph --decorate --oneline
```

`alias.hist` → creates a command "git hist" that runs the full log command

`--global` → saves the alias to ~/.gitconfig so it works in any repository on this machine

⚠ NOTE: When creating an alias, omit the "git" from the start of the aliased command. Write "log --all..." not "git log --all..."

Ignoring Files (.gitignore)

The .gitignore file tells Git which files and folders to completely ignore. Once a file matches a .gitignore pattern, Git will not suggest adding it, will not track it, and will not show it as untracked in git status.

```
# Create .gitignore
touch .gitignore

# Contents of .gitignore — one pattern per line:
.DS_Store           # specific OS file (Mac)
*.log               # wildcard — any file ending in .log
log/                # entire directory named log
*.orig              # temp files from merge resolution

# Add and commit .gitignore itself
git add .gitignore
git commit -m "Adding gitignore"
```

`.DS_Store` → excludes this specific file by name

`*.log` → the asterisk is a wildcard — matches any filename ending with .log

`log/` → the trailing slash means match a directory named log and all its contents

⚠ NOTE: .gitignore only works on UNTRACKED files. If a file was already committed to Git before being added to .gitignore, Git will continue tracking it. You must run git rm --cached <file> to stop tracking an already-committed file.

Comparing Differences (git diff)

git diff compares two states and shows what changed. The key is understanding which two sides you are comparing.

Command	Left Side	Right Side	Use Case
git diff	Staging Area	Working Directory	See unstaged changes

git diff HEAD	Last Commit	Working Directory	See ALL uncommitted changes
git diff --staged HEAD	Last Commit	Staging Area	See what will be in next commit
git diff <SHA1> <SHA2>	Commit A	Commit B	Compare two specific commits
git diff HEAD HEAD^	Last Commit	Commit before last	See what changed in last commit
git diff <branch1> <branch2>	Branch A	Branch B	Compare two branches
git diff master origin/master	Local master	Remote master	See unpushed commits

Adding -- <filename> to any git diff command limits the comparison to that specific file:

```
git diff -- README.md                # only diff the README file
git diff HEAD -- simple.html         # working dir vs last commit for one file
git difftool <args>                  # same as git diff but launches visual tool
```

`git difftool` → same syntax as git diff but opens your configured visual diff tool (e.g., P4Merge)

`HEAD^` → HEAD minus one — the commit just before the last commit

`origin/master` → the master branch as it exists on the remote (GitHub)

PHASE 4 — BRANCHING AND MERGING

Branch Basics

A branch in Git is simply a lightweight pointer to a specific commit. Creating a branch costs almost nothing. The best practice in Git is to always work on a feature branch or topic branch, never directly on master for production changes.

```
git branch                           # list local branches (* = current)
git branch -a                        # list all local + remote branches
git branch <name>                    # create a new branch (stay on current)
git checkout <name>                  # switch to a branch
git checkout -b <name>               # create AND switch in one command
git branch -m <old> <new>            # rename a branch
git branch -d <name>                 # delete a branch (must be merged)
git branch -D <name>                 # force delete (even if not merged)
```

`git branch` → with no args, lists local branches and marks current one with *

`-b` → in git checkout -b creates the branch before switching to it; equivalent to branch + checkout

`-d vs -D` → -d refuses to delete an unmerged branch; -D force-deletes regardless

`git branch -a` → shows both local branches and remote-tracking branches (e.g., origin/master)

⚠ NOTE: You cannot delete the branch you are currently on. Switch to a different branch first.

Merging — Four Scenarios

Scenario 1: Fast-Forward Merge (Happy Path)

This occurs when the target branch (master) has not had any new commits since you branched off. Git can simply move the master pointer forward to where your branch is — no merge commit needed.

```
git checkout -b title-change         # branch off master
# ... make changes ...
git commit -am "Changing title"

git checkout master                  # return to master
git diff master title-change         # review differences first
git merge title-change               # fast-forward merge
git branch -d title-change           # clean up branch
```

`git diff master title-change` → always review what you are merging before you merge it

`git merge title-change` → since master has no new commits, Git just moves its pointer forward

`Fast-forward` → the history looks linear — as if you never branched

Scenario 2: Disable Fast-Forward (--no-ff)

Sometimes you want to preserve the fact that work happened on a separate branch — even if a fast-forward was possible. The --no-ff flag forces a merge commit.

```
git checkout master
git merge --no-ff add-copyright      # force a merge commit
# editor opens for merge commit message
git log --oneline --graph --decorate # graph shows branching preserved
```

`--no-ff` → no-fast-forward; creates a merge commit even when FF is possible; preserves branch history in the graph

Scenario 3: Automatic (3-way) Merge

When both the source branch AND the target branch have had commits since they diverged, Git must do a 3-way merge. If the changes are in different files or different parts of the same file, Git can resolve this automatically and creates a merge commit.

```
# Situation: master has new commits, simple-changes has new commits
git checkout master
git merge simple-changes -m "Merging changes from simple-changes branch"
# Git auto-resolves (no conflict) and creates a merge commit
```

`-m "message"` → since a merge commit will be created, you can provide the message inline

Scenario 4: Conflicting Merge and Resolution

When both branches have modified the same lines of the same file, Git cannot auto-resolve. It marks the file with conflict markers and puts the repository into a MERGING state.

```
git merge realwork                   # triggers a conflict
# git status shows: MERGING state

# Option A: Edit file manually
# Look for <<<<<<< HEAD ... ======= ... >>>>>>> realwork markers

# Option B: Use visual mergetool
git mergetool                        # opens P4Merge or configured tool

# After resolving:
```

```
git commit -m "Done resolving merge conflicts"

# .orig files created during merge — add to .gitignore
# Add *.orig to .gitignore
git add .gitignore
git commit -m "Updating gitignore to exclude .orig files"
```

`<<<<<<< HEAD` → marks the start of changes from the current branch (target)

`=======` → divides the two conflicting versions

`>>>>>>> realwork` → marks the end of changes from the incoming branch (source)

`git mergetool` → launches the visual 3-way mergetool configured in .gitconfig

`.orig files` → Git saves backup originals of conflicted files; add *.orig to .gitignore

⚠ NOTE: During a conflict, git status shows (master|MERGING). You cannot commit regular work until the conflict is resolved and committed.

Rebasing

Rebasing is an alternative to merging for integrating changes from one branch into another. While merging creates a merge commit and preserves the branch history, rebasing replays your commits on top of another branch — producing a linear, cleaner history.

Aspect	Merge	Rebase
History shape	Branched (shows divergence)	Linear (no visible branching)
Merge commit	Creates a merge commit	No merge commit
When to use	Integrating completed features	Syncing feature branch with master
Safe for shared branches?	Yes	Never rebase shared/public branches

Simple Rebase Example

```
# On feature branch, want to incorporate master changes
git checkout myfeature
git rebase master                    # replay myfeature commits on top of master

# Now myfeature is ahead of master with a linear history
git checkout master
git merge myfeature                  # fast-forward merge — clean!
git branch -d myfeature
```

`git rebase master` → rewinds myfeature commits, fast-forwards to master, then replays myfeature commits on top

`result` → the diverged graph becomes linear; myfeature commits get new SHA-1 IDs

Aborting a Rebase

```
git rebase master                    # triggers conflict mid-rebase
git rebase --abort                   # cancel entirely; return to pre-rebase state
```

Rebase Conflict and Resolution

```
git rebase master                    # conflict detected
```

```
git mergetool                          # resolve visually
git add <file>                         # stage the resolved file
git rebase --continue                  # continue replaying remaining commits
```

`git rebase --continue` → after resolving a conflict, tells Git to continue replaying the remaining commits

`git rebase --abort` → cancels the entire rebase and restores the original state

Pull with Rebase

Instead of git pull (which merges remote changes), you can use git pull --rebase to keep your local commits on top of incoming remote changes — maintaining a linear history.

```
git fetch                              # update remote references (non-destructive)
git status                             # shows divergence between local and remote
git pull --rebase origin master        # fetch + rebase local on top of remote
```

`git fetch` → downloads remote changes and updates remote-tracking references WITHOUT merging or changing local files

`git pull --rebase` → equivalent to git fetch followed by git rebase origin/master

⚠ NOTE: git fetch is SAFE and non-destructive. git pull is git fetch + git merge (or git rebase with --rebase flag). When in doubt, fetch first, inspect, then decide how to integrate.

PHASE 5 — ADVANCED WORKFLOWS

Stashing

Stashing lets you temporarily save uncommitted work without committing it, so you can switch context (fix a bug, switch branches, etc.) and come back later. Think of it as a clipboard for work-in-progress.

```
git stash                              # save WIP; working directory becomes clean
git stash save "description"           # stash with a descriptive label
git stash list                         # list all stashes
git stash apply                        # re-apply last stash (keeps stash entry)
git stash pop                          # apply + drop last stash in one step
git stash drop                         # remove last stash entry
git stash clear                        # drop ALL stashes

# Working with specific stashes:
git stash show stash@{1}             # show what is in stash index 1
git stash apply stash@{1}            # apply a specific stash
git stash drop stash@{1}             # drop a specific stash

# Include untracked files:
git stash -u                           # stash tracked + untracked files

# Stash directly into a new branch:
git stash branch newchanges            # create branch, switch, apply stash, drop
stash
```

`git stash` → default is "git stash save"; saves WIP to a stack and restores clean working directory

`stash@{0}` → stash indices start at 0; the LAST stash created is always index 0 (like a stack)

`git stash pop` → shortcut for apply + drop; use when you are sure you want the stash and done with it

`git stash -u` → -u / --include-untracked; by default git stash ONLY stashes tracked files

`git stash branch <name>` → creates a new branch, switches to it, applies the stash, and then drops it — perfect when you realise WIP belongs on a feature branch

⚠ NOTE: Stash indices are counter-intuitive. The NEWEST stash is always {0} — like a stack (last in, first out). When you drop index {1}, index {2} becomes {1} automatically.

Stash vs Branch — When to Use Which

Use Stash When...	Use Branch When...
Quick context switch (minutes to hours)	Longer-lived feature work (days or weeks)
Not ready to commit yet	Want to share work with others via GitHub
Emergency fix needed right now	Want clear history of the separate effort
Changes are small and isolated	Changes span many files and commits

Tagging

Tags are labels attached to specific commits — used to mark significant milestones like version releases. Unlike branches, tags do not move; they permanently point to one commit.

Lightweight Tags

A lightweight tag is simply a named pointer to a commit. No extra metadata.

```
git tag myTag                        # create lightweight tag at current commit
git tag --list                       # list all tags
git show myTag                       # show the commit this tag points to
git tag --delete myTag               # delete a local tag
```

Annotated Tags (preferred for releases)

Annotated tags include the tagger's name, email, date, and a tag message — similar to a commit object. These are the ones you push to GitHub for releases.

```
git tag -a v1.0                      # opens editor for tag message
git tag v1.2 -m "Release 1.2"        # inline tag message
git tag -a v0.9-beta <commit-id>     # tag a SPECIFIC past commit
git tag -a v0.8-alpha -f <commit-id> # FORCE move an existing tag

git log --oneline --graph --decorate --all   # shows tags in history
git show v1.0                        # shows tag metadata + commit info
git diff v1.0 v1.2                   # compare two tagged versions
git difftool v1.0 v1.2
```

`git tag -a` → -a marks this as an annotated tag; Git opens your editor for the tag message

`git tag -a <name> <commit-id>` → apply a tag retroactively to any past commit

`git tag -a -f` → -f forces the tag to move to the new commit; old location is shown in output

Tags and GitHub (Remote)

```
git push origin v0.9-beta            # push a single tag to GitHub
git push origin master --tags        # push master branch + ALL tags
git push origin :v0.8-alpha          # DELETE a tag from remote (colon + tagname)
```

`git push origin --tags` → pushes all local tags that are not yet on the remote

`git push origin :<tagname>` → pushing NOTHING to that tag reference deletes it on the remote

⚠ NOTE: Tags do NOT push automatically with git push origin master. You must explicitly push tags. GitHub creates downloadable zip/tarball archives for each tag automatically.

Reset and Reflog

git reset moves the HEAD pointer to a different commit. It is powerful but can be destructive. git reflog is the safety net — it records every operation Git has performed so you can recover.

Three Types of Reset

Type	Command	HEAD Moves?	Staging Area?	Working Directory?
Soft	git reset --soft <ref>	Yes	Unchanged	Unchanged
Mixed (default)	git reset <ref>	Yes	Reset (unstaged)	Unchanged
Hard	git reset --hard <ref>	Yes	Reset	DISCARDED (destructive)

```
git reset HEAD^                    # go back 1 commit (mixed, default)
git reset HEAD~3                   # go back 3 commits
git reset --hard HEAD^             # go back 1 commit, discard all changes
git reset <commit-id>              # go to a specific commit
```

`HEAD^` → one commit behind HEAD (can also write HEAD~1)

`HEAD~3` → three commits behind HEAD

`--hard` → DANGEROUS: permanently discards uncommitted changes in staging and working directory

git reflog — The Safety Net

git reflog shows a log of every operation Git has performed — including resets, checkouts, commits, rebases — for the last 60 days. It is how you undo a bad reset.

```
git reflog                         # see all operations and their commit IDs
# Example output:
# abc123 HEAD@{0}: reset: moving to HEAD^
# def456 HEAD@{1}: commit: My last commit

git reset def456                   # go back to the commit before the bad reset
```

`HEAD@{0}` → the most recent operation in the reflog

`HEAD@{N}` → the N-th operation back in the reflog

⚠ NOTE: git reflog entries expire after 60 days. If you lose a commit, use reflog to find it within this window. After 60 days, the commits may be garbage collected.

Cherry-Pick

Cherry-pick lets you apply one specific commit from any branch onto your current branch, without merging the entire branch. The most common use case is applying a hot fix that was committed to master back into a development branch.

```
# On the branch where you want to apply the commit:
git log --oneline other-branch       # find the commit ID you want
git cherry-pick <commit-id>          # apply that specific commit here
```

`git cherry-pick <id>` → creates a new commit on the current branch with the same changes as the specified commit (but a new SHA-1)

⚠ NOTE: Cherry-pick can result in merge conflicts if the target branch has diverged significantly. Best used when commits are small, focused, and self-contained.

GitHub Collaboration Patterns

Forking and Cloning

- **Fork** — creates a personal copy of a repository in your GitHub account — you own it and can push freely
- **Clone** — downloads a fork (or any repo) to your local machine
- **origin** — automatically points to YOUR fork on GitHub
- **upstream** — convention for the original repository you forked from

Pull Requests

A Pull Request (PR) is GitHub's mechanism for proposing changes. You push a feature branch to your fork, then open a PR to the upstream repository. PRs enable code review, discussion, and comparison of differences before merging.

- **View file diffs** — the Files Changed tab in a PR shows all changes
- **Compare branches via GitHub** — comparing branches in a PR UI is the preferred visual review method
- **Merge PR** — GitHub can merge, squash, or rebase-merge

git fetch vs git pull

Command	What It Does	Modifies Local Files?	Safe?
git fetch	Downloads remote changes, updates remote-tracking references only	No	Yes — always safe
git pull	git fetch + git merge (or rebase with --rebase)	Yes	Can cause merge conflicts

PHASE 6 — INTERVIEW FAVOURITES

Critical Comparison: merge vs rebase

Dimension	git merge	git rebase

History shape	Preserves divergence; creates merge commit	Linear; no merge commit
Commit IDs	Original SHAs preserved	NEW SHA-1 IDs generated for replayed commits
Safe on shared branch?	YES	NO — never rebase a branch others are using
When to use	Completing features into master	Syncing feature branch with latest master
Reverting	Revert the merge commit	Complex — history is rewritten

Critical Comparison: git reset types

Mode	HEAD	Staging Area	Working Dir	Use When
--soft	Moves	Unchanged	Unchanged	Want to redo commit message
--mixed (default)	Moves	Cleared	Unchanged	Want to unstage and recommit
--hard	Moves	Cleared	Cleared	Want to completely undo (destructive)

Critical Comparison: git stash pop vs apply

Command	Applies Stash	Removes Stash Entry	Use When
git stash apply	Yes	No	You may want to apply the same stash again
git stash pop	Yes	Yes	Done with the stash; one-and-done

Important ⚠ Notes for Interviews

⚠ NOTE: git commit -a ONLY stages and commits TRACKED files. New files that have never been git add-ed will not be picked up by -a.

⚠ NOTE: git stash by default does NOT stash UNTRACKED files. Use git stash -u to include them.

⚠ NOTE: Branches are POINTERS, not folders. Creating a branch in Git costs essentially nothing. This is why Git encourages branching heavily — unlike older VCS systems like SVN where branches were expensive copies.

⚠ NOTE: NEVER rebase commits that have already been pushed to a shared remote branch. Rebase rewrites SHA-1 IDs, which creates a diverged history for every other developer who has pulled those commits.

⚠ NOTE: git fetch is ALWAYS safe. It only updates your remote-tracking references (like origin/master). It does not touch your local branches or working directory.

⚠ NOTE: git checkout -- <file> is IRREVERSIBLE if the file has uncommitted changes. There is no undo — the changes are gone.

⚠ NOTE: Tags do NOT automatically push. git push origin master sends commits but not tags. Use git push origin --tags or push tags individually.

⚠ NOTE: git reflog retains data for 60 days only. After that, orphaned commits may be garbage collected. Use reflog quickly after a bad reset.

⚠ NOTE: .gitignore only works on files not yet tracked. If a file was already committed, adding it to .gitignore does NOT stop Git from tracking it. Use git rm --cached <file> to stop tracking without deleting it.

⚠ NOTE: HEAD^ and HEAD~1 are equivalent. HEAD~3 means 3 commits back. HEAD^ is shorthand only for one step back.

Common Interview Questions with Key Answers

- **What is the difference between git fetch and git pull?** — fetch updates remote references only; pull does fetch + merge/rebase and modifies local branch
- **What is the staging area and why does it exist?** — it is a preparation zone between working directory and repository; lets you curate exactly which changes go into each commit
- **What is HEAD in Git?** — HEAD is a pointer that always points to the last commit on the currently active branch
- **When would you rebase instead of merge?** — when you want to keep a linear history; when syncing a feature branch with master before merging back
- **What is the difference between git reset --soft, --mixed, --hard?** — soft: moves HEAD only; mixed: moves HEAD + clears staging; hard: moves HEAD + clears staging + discards working directory changes
- **How do you resolve a merge conflict?** — git status identifies conflicted files; open them and resolve <<<... === ... >>> markers manually or via mergetool; then git add and git commit
- **What does git stash do and when would you use it?** — temporarily saves uncommitted work so you can switch context; use when interrupted mid-feature and need a clean working directory

- **What is a fast-forward merge?** — when no commits have been made on the target branch since the feature branch was created; Git simply moves the branch pointer forward with no merge commit
- **How would you apply a hotfix from master to a development branch?** — use git cherry-pick <commit-id> from the dev branch to apply just that one commit
- **What is git reflog and why is it useful?** — a log of every Git operation for 60 days; used to recover commits after an accidental reset or other history-rewriting operation

Reference Pattern Summary

```
HEAD              →  last commit on current branch
HEAD^             →  one commit before HEAD (same as HEAD~1)
HEAD~N            →  N commits before HEAD
origin            →  name of the remote repository (set by clone)
origin/master     →  the master branch on the remote
stash@{0}         →  the most recently stashed item
stash@{N}         →  the N-th stash in the list (0 = newest)
v1.0              →  a tag reference (use in diff, checkout, show)
<SHA-1>           →  direct commit reference (first 6-7 chars usually
sufficient)
```

Complete Command Quick Reference

Command	Purpose
git init	Initialise a new repository
git clone <url>	Clone a remote repository locally
git status	Show working directory and staging area state
git add <file>	Stage a file
git add .	Stage all files recursively
git add -A	Stage all changes including renames/deletions
git commit -m "msg"	Commit with inline message
git commit -am "msg"	Stage tracked files and commit
git commit --amend	Modify the last commit
git pull origin master	Fetch + merge from remote
git push origin master	Push commits to remote
git fetch	Update remote references only
git log --oneline --graph --decorate --all	Visual commit history
git diff	Working dir vs staging area
git diff HEAD	Working dir vs last commit
git diff --staged HEAD	Staging area vs last commit
git branch -b <name>	Create and switch to new branch

git checkout <branch>	Switch to existing branch
git merge <branch>	Merge branch into current
git merge --no-ff <branch>	Merge preserving branch history
git rebase master	Replay current branch commits on top of master
git rebase --abort	Cancel in-progress rebase
git rebase --continue	Continue rebase after resolving conflict
git stash	Save WIP and clean working directory
git stash pop	Apply + drop last stash
git stash -u	Stash including untracked files
git tag -a <name> -m "msg"	Create annotated tag
git push origin --tags	Push all tags to remote
git reset HEAD <file>	Unstage a file
git reset --hard HEAD	Discard all uncommitted changes
git checkout -- <file>	Discard working dir changes for one file
git reflog	View log of all Git operations (60-day window)
git cherry-pick <SHA>	Apply a single commit to current branch
git ls-files	List all tracked files
git mergetool	Launch visual conflict resolution tool
git help <command>	Open manual for any Git command